

IA agentique pour le développement logiciel

Concepts de base pour travailler avec un agent de codage.

Animé par Jordi Bolibar et Julien Le Sommer

01 — POURQUOI CETTE SESSION

Trois objectifs pour 3 h de travail

Comprendre

les concepts de base

Pratiquer

sur un problème guidé, pas à pas

Démarrer

sur un vrai problème de développement

Apprendre à coder avec des agents, de manière robuste et fiable.

Méta : ces slides — et le dépôt qui les héberge — ont été produites avec un agent.

02 — UN LLM, C'EST QUOI ?

Une machine à prédire le mot suivant

texte → tokens → prédiction du token suivant

Un token ≈ un fragment de mot (4 caractères en moyenne) — l'unité que le modèle lit et prédit.

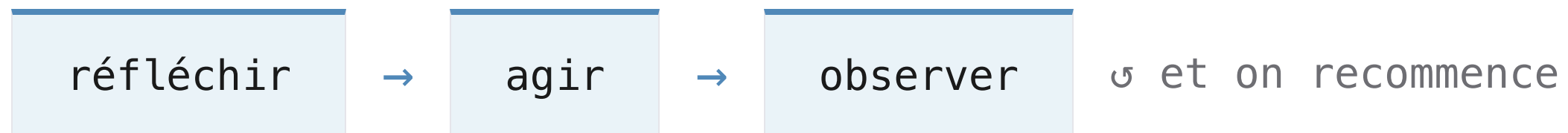
- Entraîné sur un corpus immense : des capacités larges, aucune garantie de vérité
- Il peut se tromper avec aplomb — les hallucinations sont plausibles
- Pas de mémoire entre deux appels : tout passe par la fenêtre de contexte, finie

D'où la règle : la vérification reste humaine.

03 — UN AGENT, C'EST QUOI ?

Un LLM dans une boucle, avec des outils

LLM + instructions (rôle, tâche) + contexte / mémoire + outils (fichiers, shell, git)



Le chat conseille

vous tapez, vous exécutez

L'agent agit

il écrit, exécute, itère — **vous gardez la supervision**

04 — EN PRATIQUE

Deux ingrédients : un modèle, un harnais

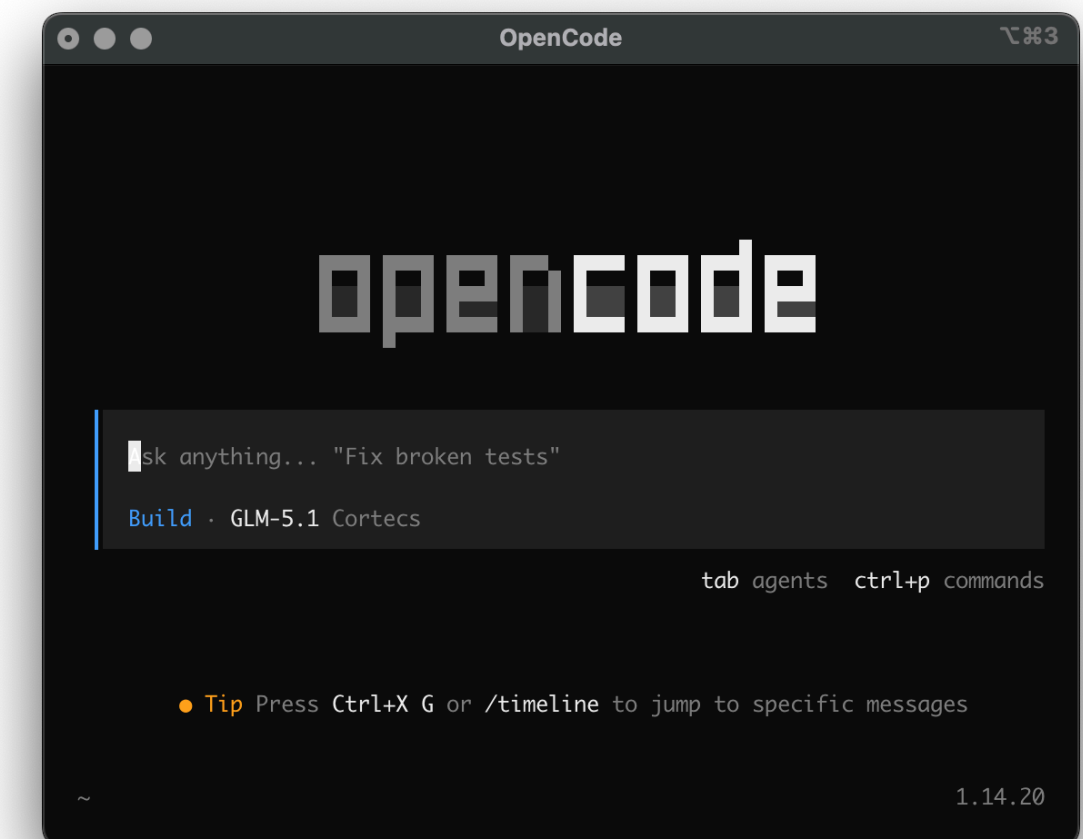
L'ACCÈS AUX MODÈLES

- API / SDK + clé d'API
- facturation au token ou forfaitaire
- Cortecs — passerelle européenne, RGPD

LE HARNAIS

- VS Code + Kilo Code — l'IDE
- OpenCode — le terminal
- aussi : Copilot, Claude Code, Continue...

Notre choix aujourd'hui : OpenCode · VS Code + Kilo Code



05 — GESTION DU CONTEXTE

Le contexte est la ressource critique

Du prompt engineering au context engineering — nourrissez-le, économisez-le.



Chaque appel consomme un **budget fixe de tokens** — la fenêtre de contexte.

Compaction

résumer les longues sessions, garder l'essentiel

Notes structurées

AGENTS.md, PLANS.md, DESIGN.md — créés via /init

Le bon niveau de détail

un plan ni trop vague, ni trop verbeux

Sous-agents

découper la tâche, spécialiser, cloisonner le contexte

AGENTS.md

Commandes

- tests : `pytest -v`
- lint : `ruff check .`

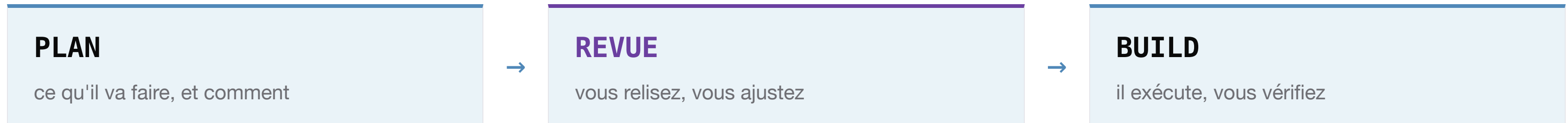
Règles

-  toujours relire le diff
-  demander avant un changement lourd
-  jamais de secrets

un écran, pas un manuel — vivant, itéré

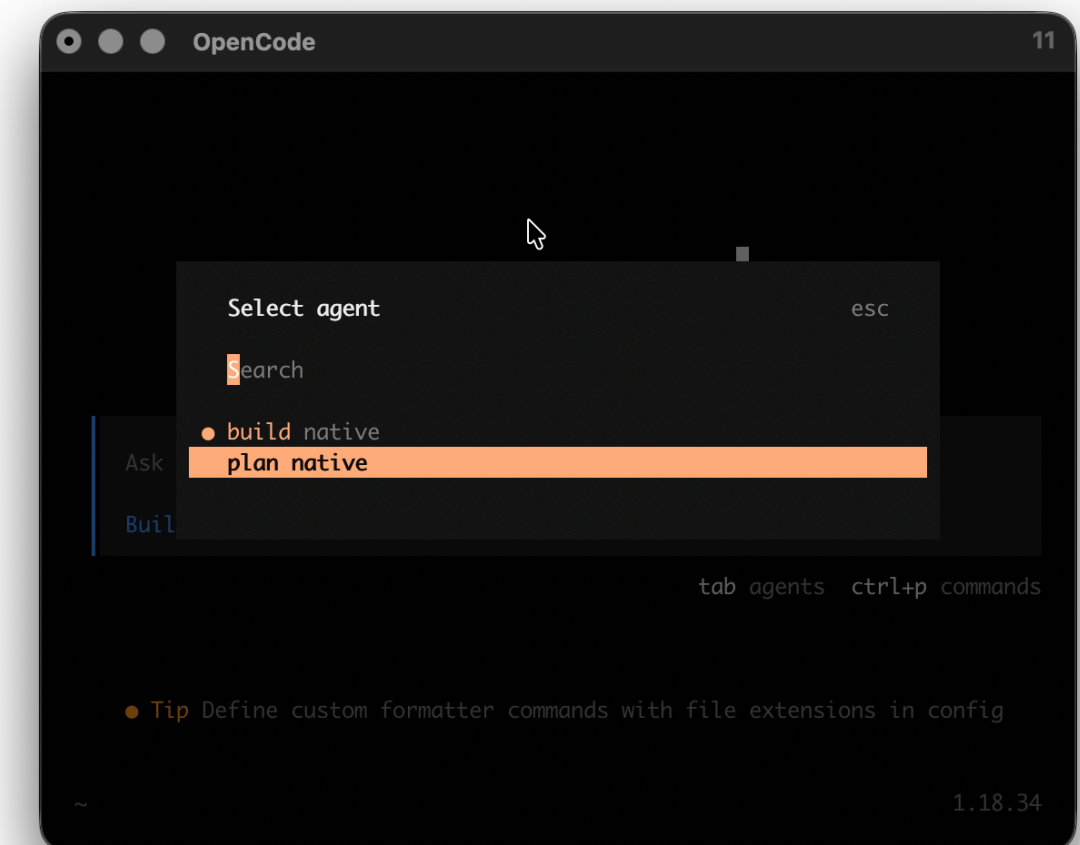
06 — LE WORKFLOW TYPIQUE

Trois temps, une boucle



réfléchir → agir → observer

Parlez-lui comme à un·e collègue junior : contexte précis, petites tâches, feedback régulier.



Six réflexes avant de commencer

- ✓ **Petites tâches** — découper, valider, recommencer
- ✓ **Vérifier** — lire le code, questionner le surprenant
- ✓ **Tests** — les exécuter avant de continuer
- ✓ **Hygiène git** — commits fréquents, diffs relus
- ✓ **Coûts** — choisir le plus petit modèle possible pour la tâche
- ✓ **Permissions** — lire avant d'approuver, jamais de secrets

La vérification reste votre responsabilité.

L'atelier d'aujourd'hui

Deux temps de 60 minutes, séparés par une pause — restitution collective en clôture.

#	DURÉE	ÉTAPE	SUPPORT
1	15 min	Introduction — concepts de base : LLM, agent, contexte, workflow	slides-introduction/
2	15 min	Mise en place technique : Cortecs, puis OpenCode ou VS Code + Kilo Code	SETUP.md
3	60 min	Atelier guidé — le climat du Poët-Laval, dont 15 min de débrief	poet-laval-climate
4	15 min	Pause	—
5	60 min	Travail libre — votre vrai problème, avec plans et AGENTS.md	—
6	15 min	Restitution — chacun·e décrit son travail avec l'agent	—

Trois choses à retenir

Agent = LLM + instructions + contexte + outils

Le contexte est la ressource clef à contrôler

Petites tâches, plan avant build, hygiène git